

University of Exeter

MSc Data Science

MSc Research Project Proposal

LLM Hallucination Detection and Verification

Student: Yuangeng Zhang

Module: COMM514

Supervisor: Prof. Shiqiang Wang

Abstract

The abstract will be completed once the research question, methodology and expected contribution have been finalised.

Generative AI declaration

To be completed in accordance with the COMM514 assessment brief.

1 Introduction

Large language models (LLMs) are now used for tasks ranging from question answering and information retrieval to writing and decision support. Their fluency, however, can make incorrect information difficult to recognise. A model may produce an answer that is coherent and confident while containing claims that are unsupported or factually wrong. These outputs are commonly described as hallucinations, and they remain an important obstacle to the reliable use of LLMs, particularly when users cannot easily verify the information themselves [1].

One way of addressing this problem is to ask the model, or another language model, to check a generated answer after it has been produced. This is attractive because it can be applied to black-box models without access to their internal parameters. SelfCheckGPT, for example, detects possible hallucinations by looking for disagreement across multiple samples from the same model [2]. More generally, LLMs have also been used as judges of other model outputs, although such judging is not free from bias. Previous work has identified effects such as position and verbosity bias, as well as cases in which a judge appears to favour outputs associated with itself [3]. There is therefore a basic difficulty in relying on the same model both to produce an answer and to assess whether that answer should be trusted.

Evidence on intrinsic self-correction raises a similar concern. Huang et al. found that asking an LLM to reconsider its own reasoning without external feedback did not reliably improve performance and could sometimes make an initially correct response worse [4]. Other verification-based approaches have shown better results when some degree of independent information is introduced. Chain-of-Verification, for instance, separates verification questions from the original response in order to reduce the influence of an existing hallucination [5]. More recent work has gone further by using a separate language model as a verifier. Xue et al. showed that cross-model consistency can provide useful information beyond self-consistency, while also demonstrating that the benefit depends on how and when the additional verifier is used [6]. FINCH-ZK similarly uses responses from different model families to identify fine-grained factual inconsistencies and reports substantial improvements over single-model baselines on hallucination detection benchmarks [7].

These findings make cross-model verification promising, but they do not establish that a second model necessarily provides an independent check. Different LLMs can share training sources, model families and broader patterns of learned behaviour. Kim et al., in a large-scale study of more than 350 models, found substantial correlation between model errors and showed that such correlation can remain strong even among relatively capable models from different architectures or providers [8]. This creates an important practical question for hallucination detection. If a generator and its verifier tend to make the same mistakes, agreement between them may create confidence without adding much genuine evidence of correctness. Conversely, some generator-verifier combinations may provide more complementary information than others.

This project will therefore investigate the reliability of self- and cross-model verification for factual LLM outputs, with particular attention to the choice of generator-verifier pairing and the errors shared between models. Rather than assuming that cross-model verification is inherently superior, the study will examine when a second model provides useful independent evidence and when the two models fail together. The current research question is:

To what extent does cross-model verification improve hallucination detection over self-verification, and how is this improvement affected by generator-verifier pairing and correlated model errors?

The project will use a reproducible black-box experimental setting in which one LLM first generates answers to factual questions and those same answers are subsequently assessed through self-verification and verification by other models. Performance will be compared against known reference answers, alongside an analysis of cases in which the generator and verifier agree on the same error. This provides a manageable way to study not only whether cross-model verification works, but also the conditions under which it can reasonably be treated as an independent reliability check.

2 Background and Related Work

2.1 Hallucination Detection in Factual Question Answering

Hallucination broadly refers to generated content that is unsupported by available evidence or conflicts with established facts [1]. This project focuses specifically on factual question answering, where a generated answer can be checked against a known reference or other reliable evidence. This narrower setting allows verification methods to be evaluated against external ground truth rather than relying only on another model’s judgement.

Black-box hallucination detection is particularly relevant because many deployed LLMs do not expose internal weights, hidden states or token probabilities. SelfCheckGPT addresses this by sampling several responses from the same model and measuring whether factual claims remain consistent across generations [2]. The underlying idea is that reliable knowledge should be relatively stable, while hallucinated claims are more likely to vary.

This approach is useful, but consistency does not necessarily imply correctness. Repeated generations come from the same model and may reproduce the same misconception. Self-consistency can therefore provide an uncertainty signal without guaranteeing independent evidence of factual truth.

2.2 From Self-Refinement to Self-Verification

The broader idea of asking an LLM to examine its own output has been explored in several forms. Self-Refine uses the same model to generate an answer, critique it and revise it iteratively, and reports improvements across a range of tasks without additional training [9]. This suggests that self-feedback can be useful when the model is able to identify weaknesses in its initial response.

The benefit is not universal. Huang et al. found that intrinsic self-correction in reasoning tasks did not reliably improve performance and could sometimes change an initially correct answer into an incorrect one [4]. Related concerns appear in LLM-based judging: Zheng et al. showed that model judgements can be affected by systematic biases, including response position and verbosity [3]. These findings suggest that a model’s ability to produce an answer does not automatically make it a neutral or reliable verifier of that answer.

Other methods introduce information that is more independent from the original generation. Chain-of-Verification separates the initial response from later verification questions before revising the answer [5]. CRITIC allows the model to consult external tools such as search engines or code interpreters during the critique process [10]. Together, these approaches indicate that verification reliability may depend not only on the act of checking an answer, but also on where the verification signal comes from.

2.3 Cross-Model Verification

Cross-model verification replaces or supplements self-checking with a second LLM. The generator first produces an answer, while another model assesses whether that answer should be trusted. This can potentially introduce information that is unavailable when the generator only re-examines its own output.

Xue et al. compared self-consistency and cross-model consistency in black-box hallucination detection and found that an additional verifier can provide complementary information beyond repeated sampling from a single model [6]. Their two-stage procedure also showed that external verification can be applied selectively to uncertain cases rather than to every response.

FINCH-ZK similarly uses outputs from different model architectures and semantically equivalent prompts to identify fine-grained factual inconsistencies [7]. Its results support cross-model consistency as a practical hallucination-detection strategy. These studies establish that a second model can be useful, but they leave open a more specific question: whether that benefit depends on which generator and verifier are paired.

A verifier may help because it is more capable, because it has a different error profile, or both. Conversely, simply using a different model does not guarantee that its judgement is independent of the generator.

2.4 Correlated Errors and Generator–Verifier Pairing

The assumption of independence is important when agreement between models is treated as evidence of correctness. LLMs may share training sources, architectural families, optimisation practices or providers, and different systems can therefore make similar mistakes.

Kim et al. analysed more than 350 LLMs and found substantial correlation in model errors [8]. On one evaluated leaderboard dataset, when two models were both wrong, they produced the same error in approximately 60% of those cases. Shared architecture and provider were associated with stronger correlation, although substantial overlap could remain even across otherwise different models.

This has a direct consequence for hallucination verification. If both generator and verifier share the same factual misconception, the verifier may endorse an incorrect answer rather than expose it. A different pairing may provide more complementary evidence if the models’ failure patterns are less strongly aligned.

Existing work therefore points to two findings that need to be considered together: cross-model verification can improve hallucination detection [6, 7], while model errors themselves can be correlated [8]. The present project builds on this tension by examining how verification performance changes across generator–verifier pairings and whether shared factual errors help explain those differences.

3 Research Gap, Aim and Research Questions

3.1 Research Gap

Recent studies show that cross-model signals can improve hallucination detection beyond repeated sampling from a single model [6, 7]. However, the use of a second model does not guarantee an independent judgement. Errors across LLMs can be correlated, particularly when models share related training sources, architectures or providers [8]. If a generator and verifier fail on the same factual questions, agreement between them may add little evidence of correctness.

Existing work has therefore established two relevant findings: cross-model verification can provide useful complementary information, while model errors themselves may overlap. What remains less clear is how strongly verification performance depends on the specific generator–verifier pairing, and whether shared factual errors help explain why some pairings provide greater benefit than others.

3.2 Aim and Research Questions

The aim of this project is to evaluate the reliability of self- and cross-model verification for factual LLM outputs, with particular attention to generator–verifier pairing and shared factual errors.

The study will address three questions:

RQ1: To what extent does cross-model verification improve hallucination detection over self-verification in LLM-generated factual answers?

RQ2: How does hallucination-detection performance vary across different generator–verifier pairings?

RQ3: How is cross-model verification performance related to the overlap in factual errors between generator and verifier models?

The questions do not assume that cross-model verification will always outperform self-verification. Cases in which a second model provides little benefit or introduces additional errors are also relevant to understanding when external verification is reliable.

3.3 Objectives and Expected Contribution

The project will establish a reproducible factual question-answering setting in which the same generated answers are assessed through self-verification and verification by other models. Performance will be compared across generator-verifier pairings using independently established ground truth, while error-level analysis will examine hallucinations missed by both approaches, cases detected only by an external verifier, and correct answers incorrectly rejected.

A further objective is to compare the factual error profiles of the models and determine whether stronger error overlap is associated with weaker cross-model verification. The expected contribution is therefore a controlled empirical account of when a second model provides useful complementary evidence and when shared failure patterns limit that benefit. Any practical recommendation on verifier selection will be based on consistent results across multiple pairings rather than on a single model comparison.

4 Methodology and Experimental Design

4.1 Experimental Overview

The study will use a controlled black-box experiment in which language models are separated into two roles: generator and verifier. A generator will first answer a set of factual questions without access to external search or retrieval tools. The generated answer will then be assessed twice: once by the generating model itself and once by a different model. This creates a direct comparison between self-verification and cross-model verification while keeping the underlying answer fixed.

The same verification prompt and output labels will be used in both conditions. The verifier will receive the original question and the generated answer, but not the benchmark reference answer. Each verification will also be performed in a fresh context so that the self-verification condition does not retain the generator’s earlier conversation or reasoning. This is intended to isolate the effect of verifier identity rather than differences in prompt content or conversational history.

4.2 Benchmark and Ground-Truth Labelling

The main experiment will use a short-form factual question-answering benchmark. SimpleQA was designed specifically for this setting and contains 4,326 fact-seeking questions with short, independently supported reference answers [11]. Its questions were intentionally constructed to remain challenging for strong models, making it useful for eliciting factual errors rather than evaluating only questions that current models already answer reliably.

For the primary evaluation, this project will use SimpleQA Verified [12]. This version retains 1,000 questions from SimpleQA after additional de-duplication, topic balancing and reconciliation of problematic reference answers. The smaller and cleaner set is more suitable for the present study because errors in benchmark labels could otherwise be mistaken for failures of either the generator or verifier.

Each generator will answer the same set of questions under a closed-book condition. Generated responses will be assigned one of three ground-truth outcome categories: *correct*, *incorrect*, or *not attempted*, following the benchmark’s grading convention. Incorrect attempted answers will form the positive hallucination cases for the verification experiment, while correct answers will form the negative cases. Abstentions will be reported separately rather than treated as hallucinations.

Ground-truth grading will be based on the benchmark reference answer and grading rubric rather than on the experimental verifier being evaluated. Where automatic grading produces an ambiguous result, the question, reference answer and model response will be reviewed separately. This separation is important because using the same model both as experimental verifier and as the source of the ground-truth label would make the comparison circular.

4.3 Model Selection and Generator–Verifier Pairings

The experiment will use models from more than one model family so that cross-model verification is not limited to different instances of closely related systems. The current implementation uses Ollama Cloud, which allows the same Python interface to be retained while avoiding local hardware limitations. Candidate model families currently include Gemma, Qwen and GPT-OSS. The exact model tags used in the main experiment will be fixed before the full evaluation and recorded together with the date and generation settings.

The initial design will use three model families. Each model will first generate answers to the benchmark questions, producing an independent factual-error profile. The same models will then act as verifiers. For a response generated by Model A, the self-verification condition will use Model A as verifier, while the cross-model conditions will use Models B and C. Repeating this process with each model as generator gives both self-verification results and several ordered generator–verifier pairings.

This design makes it possible to distinguish verifier capability from pairing effects. A verifier that performs well for one generator may not provide the same improvement for another, and an external verifier need not be stronger overall in order to contribute complementary information.

4.4 Verification Protocol

For every generated answer, the verifier will be given the factual question and candidate answer and asked to decide whether the answer is factually correct. The prompt wording will be held constant across self- and cross-model conditions, and the reference answer will remain hidden from the verifier.

The primary verifier output will be a binary judgement indicating whether the candidate answer should be accepted or rejected. Responses that cannot be parsed into the required judgement will be retained and reported rather than silently discarded. Generation and verification settings, including temperature and other sampling parameters supported by the model interface, will be fixed within each stage of the experiment.

To avoid adapting the protocol to the final evaluation results, a small development subset will first be used to test prompt wording, output parsing and data handling. Once the procedure has been fixed, the remaining questions will be treated as held-out evaluation data. The random split and all processing steps will be recorded so that the same experiment can be reproduced.

4.5 Shared-Error Analysis and Reproducibility

In addition to evaluating verifier judgements, the study will record the factual errors made when each model acts as a generator. For each pair of models, the set of questions answered incorrectly by both models will be compared with the questions answered incorrectly by either model alone. This will provide a direct measure of how strongly their factual failures overlap.

The overlap analysis will then be considered alongside verification performance. In particular, the study will examine whether generator–verifier pairings with more strongly overlapping factual errors are also more likely to miss the same hallucinated answers. This analysis is intended to address RQ3 without assuming in advance that error overlap must explain all differences between pairings.

All questions, generated answers, verifier decisions, model identifiers, experimental settings and timestamps will be stored in structured result files. The Python scripts used for data preparation, generation, verification and evaluation will be maintained in the existing GitHub repository. Raw benchmark files will remain separate from processed outputs, and experiment configurations will be fixed before the final runs. These steps are intended to keep the comparison traceable and reproducible while allowing the model and benchmark choices to be revised during the pilot stage.

5 Evaluation and Success Criteria

Verification performance. The factual performance of each generator will first be reported as the proportions of *correct*, *incorrect* and *not attempted* responses. For hallucination detection, only attempted answers will enter the main binary evaluation: incorrect answers will be treated as positive cases and correct answers as negative cases. Accuracy, precision, recall, F1-score and false-positive rate will be reported, while abstentions and unparseable verifier outputs will be kept separate.

The class distribution will be reported alongside these metrics. This is important because a preliminary 20-question pilot contained incorrect answers and abstentions but no correct attempted answers, making some measures, especially precision and false-positive behaviour, uninformative. The final experiment will therefore use a larger evaluation set and will interpret performance only in the context of the observed class balance.

Paired and error-level analysis. Self- and cross-model verification will be compared on exactly the same generated answers. Each case will therefore fall into one of four categories: both verifiers correct, self only correct, cross-model only correct, or both wrong. Disagreement cases will be examined directly rather than being hidden within aggregate scores. Where the sample size is sufficient, McNemar’s test and confidence intervals will be used to assess whether observed differences are likely to reflect more than case-level variation.

For RQ2, results will be reported separately for each ordered generator–verifier pairing, with improvement over self-verification calculated within each generator. For RQ3, the factual error sets produced by different models will be compared using Jaccard similarity. These overlap values will then be considered alongside verification performance to examine whether stronger shared-error patterns coincide with weaker cross-model benefit.

Success criteria. Project success will not depend on cross-model verification outperforming self-verification. The study will be considered successful if it provides a reproducible comparison across multiple generator–verifier pairings using independently established ground truth, and if the analysis can explain where self- and cross-model verification agree, where they differ, and how those differences relate to shared factual errors.

A stronger outcome would be the identification of consistent pairing effects that could inform verifier selection in black-box systems. Such conclusions will only be drawn if they are supported across more than one pairing and are not driven by a small number of individual cases.

6 Risks, Ethics and Limitations

Risks and mitigation. The main practical risk is dependence on remotely hosted black-box models. Cloud availability, usage limits and model versions may change during the project, and individual requests can occasionally fail or take unusually long to complete. Exact model identifiers, dates and generation settings will therefore be recorded for each experiment. Requests will use timeouts and limited retries, while intermediate results will be saved incrementally so that interrupted runs can be resumed without repeating completed cases. A small development run will also be completed before each full experiment to check model access, response format and output parsing.

A second risk concerns the quality and balance of the evaluation data. Benchmark errors or ambiguous reference answers could lead to incorrect ground-truth labels, while an evaluation set containing very few correct or incorrect generated responses would make some verification metrics difficult to interpret. Reference-based grading will therefore be kept separate from the experimental verifiers, ambiguous cases will be reviewed, and the distribution of correct, incorrect and abstained responses will be reported before performance measures are interpreted.

Ethical considerations. No human participants will be recruited and no new personal or sensitive data will be collected. The study will use publicly released benchmark questions and model-generated responses, subject to the relevant dataset licences and service terms. Generated text may occasionally contain inaccurate, biased or otherwise undesirable material; such outputs will be retained only where necessary for analysis and will not be presented as factual information. Experiment files will be stored within the project workspace and only material required for reproducibility will be committed to the repository.

Limitations. The study is intentionally limited to factual question answering and a small number of black-box LLM families, so its findings may not generalise to longer-form generation, retrieval-augmented systems or models with substantially different capabilities. Differences between generator–verifier pairings may also reflect verifier capability as well as differences in error patterns. For this reason, pairing effects will be interpreted alongside each model’s independent factual performance rather than being attributed to error correlation alone. Finally, because hosted models can change over time, the experiments provide a reproducible record of the tested configurations rather than a claim that identical outputs will remain obtainable indefinitely.

7 Project Plan and Deliverables

The remaining work will proceed in four stages. The first stage will finalise the benchmark, model set and verification protocol using small development runs. The second will run the main generation and verification experiments, with all model settings and intermediate outputs recorded. The third will evaluate self- and cross-model verification across generator–verifier pairings and complete the shared-error analysis. The final stage will consolidate the results, complete the dissertation and carry out reproducibility checks before submission.

Stage	Main work	Output
1	Finalise benchmark, model pairings and prompts; complete development tests	Fixed experiment protocol
2	Generate benchmark answers and obtain independent ground-truth labels	Generation dataset and labels
3	Run self- and cross-model verification; analyse pairing and shared-error effects	Evaluation results and case analysis
4	Interpret results, complete dissertation and reproduce key experiments	Final dissertation and reproducible project repository

The main deliverables will therefore be the dissertation, a version-controlled Python implementation of the experimental pipeline, structured experimental results, and sufficient configuration information to reproduce the principal analyses. The existing pilot work will be retained as preliminary evidence of feasibility but will remain separate from the final evaluation.

References

- [1] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, 2025.
- [2] Potsawee Manakul, Adian Liusie, and Mark Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9004–9017, Singapore, 2023. Association for Computational Linguistics.

- [3] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [4] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations*, 2024.
- [5] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-verification reduces hallucination in large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 3563–3578, Bangkok, Thailand, 2024. Association for Computational Linguistics.
- [6] Yihao Xue, Kristjan Greenewald, Youssef Mroueh, and Baharan Mirzasoleiman. Verify when uncertain: Beyond self-consistency in black box hallucination detection. *arXiv preprint arXiv:2502.15845*, 2025.
- [7] Aman Goel, Daniel Schwartz, and Yanjun Qi. Zero-knowledge LLM hallucination detection and mitigation through fine-grained cross-model consistency. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1982–1999, Suzhou, China, 2025. Association for Computational Linguistics.
- [8] Elliot Myunghoon Kim, Avi Garg, Kenny Peng, and Nikhil Garg. Correlated errors in large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 30038–30066. PMLR, 2025.
- [9] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Prasad Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- [10] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, 2024.
- [11] Jason Wei, Karina Nguyen, Hyung Won Chung, Yunxin Joy Jiao, Spencer Papay, Amelia Glaese, John Schulman, and William Fedus. Measuring short-form factuality in large language models. *arXiv preprint arXiv:2411.04368*, 2024.
- [12] Lukas Haas, Gal Yona, Giovanni D’Antonio, Sasha Goldshtein, and Dipanjan Das. SimpleQA Verified: A reliable factuality benchmark to measure parametric knowledge. *arXiv preprint arXiv:2509.07968*, 2025.